

Efficient Multi-Valued Heuristic Search via KD-Tree Dominance Pruning

Anonymous submission

Abstract

1 Introduction

2 Related Work

3 Problem Formulation

We consider the *multi-objective shortest-path* (MOSP) problem on a directed graph $\mathcal{G} = \langle S, E, \mathbf{c} \rangle$, where S is a finite set of states, $E \subseteq S \times S$ is a set of directed edges, and

$$\mathbf{c}: E \rightarrow \mathbb{R}_{\geq 0}^d$$

assigns each edge a d -dimensional non-negative cost vector. A MOSP instance is $\mathcal{P} = \langle S, E, \mathbf{c}, s_{\text{start}}, s_{\text{goal}} \rangle$, where s_{start} and s_{goal} are the start and goal states.

A path $\pi = \langle s_1, \dots, s_k \rangle$ has cost

$$\mathbf{c}(\pi) = \sum_{i=1}^{k-1} \mathbf{c}(s_i, s_{i+1}).$$

For two cost vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}_{\geq 0}^d$, we say that $\mathbf{x}$ *weakly dominates* $\mathbf{y}$, denoted $\mathbf{x} \preceq \mathbf{y}$, if $x_i \leq y_i$ for all $i = 1, \dots, d$. We say that $\mathbf{x}$ *dominates* $\mathbf{y}$, denoted $\mathbf{x} \prec \mathbf{y}$, if $\mathbf{x} \preceq \mathbf{y}$ and the inequality is strict in at least one component. Two vectors are mutually non-dominated if neither dominates the other. For a set X of cost vectors, let $\text{ND}(X)$ denote its cost-unique subset of mutually non-dominated vectors.

A *solution* is a path from s_{start} to s_{goal} . The objective of MOSP is to compute the *Pareto-optimal solution set*, i.e., all solutions whose cost vectors are not dominated by the cost vector of any other solution.

Multi-valued heuristics. Rather than associating each state with a single heuristic vector, we consider a *multi-valued heuristic* (MVH)

$$H: S \rightarrow 2^{\mathbb{R}_{\geq 0}^d},$$

where $H(s)$ is a finite set of mutually non-dominated heuristic vectors.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

An MVH is *admissible* if, for every state s and every path π from s to s_{goal} , there exists some $\mathbf{h} \in H(s)$ such that

$$\mathbf{h} \preceq \mathbf{c}(\pi).$$

Consider a search path reaching state s with accumulated cost vector $\mathbf{g}$. Each heuristic vector $\mathbf{h} \in H(s)$ induces an evaluation vector

$$\mathbf{f} = \mathbf{g} + \mathbf{h}.$$

Thus, the path is associated with the set

$$F(s, \mathbf{g}) = \text{ND}(\{\mathbf{g} + \mathbf{h} \mid \mathbf{h} \in H(s)\}).$$

Dimensionality reduction. Following dimensionality-reduction approaches for multi-objective search, we define the truncation operator

$$\text{Tr}(\mathbf{x}) = (x_2, \dots, x_d),$$

which removes the first component of a cost vector. For a set X , we write

$$\text{Tr}(X) = \text{ND}(\{\text{Tr}(\mathbf{x}) \mid \mathbf{x} \in X\}).$$

Consequently, dominance tests that exploit dimensionality reduction operate in $d-1$ dimensions. In particular, for a fixed path-cost vector $\mathbf{g}$, the truncated evaluation corresponding to a heuristic $\mathbf{h} \in H(s)$ is

$$\text{Tr}(\mathbf{g} + \mathbf{h}) = \text{Tr}(\mathbf{g}) + \text{Tr}(\mathbf{h}).$$

Our goal is to exploit the geometric structure of $\text{Tr}(H(s))$ to accelerate these dominance tests.

4 Background: Lazy Multi-Valued Heuristic Search

Our approach builds on L-NAMOA_{dr}^{*}-MVH (Wolff, Felner, and Salzman 2026), a multi-objective search algorithm that combines multi-valued heuristics with dimensionality reduction while requiring only an admissible MVH. The key idea of the algorithm is to avoid explicitly creating a separate search node for every combination of a path cost $\mathbf{g}$ and a heuristic vector $\mathbf{h} \in H(s)$. Instead, each physical path is represented by a single search node that is lazily associated with one heuristic vector at a time.

A search node n reaching state s stores its path-cost vector $\mathbf{g}(n)$ and a currently selected heuristic vector $\mathbf{h}(n) \in H(s)$. Its evaluation vector is

$$\mathbf{f}(n) = \mathbf{g}(n) + \mathbf{h}(n).$$

OPEN is ordered lexicographically according to these evaluation vectors.

For each state s , the algorithm maintains two closed sets. The full closed set $G_{cl}(s)$ contains path-cost vectors $\mathbf{g}$ discovered for s , while the truncated closed set $G_{cl}^{Tr}(s)$ contains their truncated counterparts used for dimensionality-reduced dominance tests.

Algorithm 1 summarizes the search at a high level; the complete algorithm is given as Algorithm 2 by Wolff et al. (Wolff, Felner, and Salzman 2026).

Algorithm 1 L-NAMOA^{*}_{dr}-MVH (high level)

Input: MOSP instance $\mathcal{P}$; admissible MVH H

Output: Pareto-optimal solution set

```

1: Create root node  $n$  at  $s_{start}$  with  $\mathbf{g}(n) = \mathbf{0}$  and  $\mathbf{h}(n) = \min_{lex} H(s_{start})$ 
2:  $\mathbf{f}(n) \leftarrow \mathbf{g}(n) + \mathbf{h}(n)$ 
3: OPEN  $\leftarrow \{n\}$ 
4: Initialize  $G_{cl}(s)$  and  $G_{cl}^{Tr}(s)$  for all  $s$ 
5: Sols  $\leftarrow \emptyset$ 
6: while OPEN is not empty do
7:    $n \leftarrow$  node in OPEN with lexicographically smallest  $\mathbf{f}(n)$ 
8:   if  $\text{Tr}(\mathbf{f}(n))$  is weakly dominated by  $G_{cl}^{Tr}(s_{goal})$  then
9:      $\mathbf{h} \leftarrow \text{CHOOSEH}(s(n), \mathbf{g}(n), G_{cl}^{Tr}(s_{goal}))$ 
10:    if  $\mathbf{h} \neq \perp$  then
11:       $\mathbf{h}(n) \leftarrow \mathbf{h}$ 
12:       $\mathbf{f}(n) \leftarrow \mathbf{g}(n) + \mathbf{h}$ 
13:      Reinsert  $n$  into OPEN
14:    continue
15:   if LOCALDOMCHECK( $s(n), \mathbf{g}(n)$ ) then
16:     continue
17:   Update  $G_{cl}(s(n))$  and  $G_{cl}^{Tr}(s(n))$ 
18:   if  $s(n) = s_{goal}$  then
19:     Sols  $\leftarrow$  Sols  $\cup \{\mathbf{g}(n)\}$ 
20:   continue
21:   for all successors  $s'$  of  $s(n)$  do
22:      $\mathbf{g}' \leftarrow \mathbf{g}(n) + \mathbf{c}(s(n), s')$ 
23:      $\mathbf{h}' \leftarrow \text{CHOOSEH}(s', \mathbf{g}', G_{cl}^{Tr}(s_{goal}))$ 
24:     if  $\mathbf{h}' = \perp$  then
25:       continue
26:     if LOCALDOMCHECK( $s', \mathbf{g}'$ ) then
27:       continue
28:     Create node  $n'$  with  $\mathbf{g}(n') = \mathbf{g}', \mathbf{h}(n') = \mathbf{h}'$ ,
    and  $\mathbf{f}(n') = \mathbf{g}' + \mathbf{h}'$ 
29:     Insert  $n'$  into OPEN
30: return Sols

```

Lazy heuristic selection and re-evaluation. CHOOSEH scans the heuristic vectors in $H(s)$ in lexicographic order and selects the first vector whose truncated evaluation is not dominated by the current truncated goal frontier.

Let

$$T = G_{cl}^{Tr}(s_{goal})$$

denote the current truncated goal frontier. Then CHOOSEH can be written conceptually as

$$\mathbf{h}^* = \min_{lex} \{\mathbf{h} \in H(s) \mid \nexists \mathbf{t} \in T: \mathbf{t} \preceq \text{Tr}(\mathbf{g} + \mathbf{h})\}.$$

If no such heuristic exists, CHOOSEH returns $\perp$.

As shown in Algorithm 1, CHOOSEH is invoked in two situations. First, when a successor path is generated, CHOOSEH selects the heuristic used to evaluate that path. If no usable heuristic exists, the path is discarded; otherwise, it is subjected to the algorithm's local dominance check and, if it survives, inserted into OPEN according to $\mathbf{f} = \mathbf{g} + \mathbf{h}$. Second, when a node is extracted from OPEN and its current evaluation has become dominated by the goal frontier, CHOOSEH is invoked again to select the next lexicographically smallest usable heuristic. If one exists, the node is reinserted into OPEN with its updated evaluation vector; otherwise, it is discarded.

Thus, the algorithm implicitly considers multiple heuristic vectors for the same path without simultaneously maintaining a separate OPEN node for every $(\mathbf{g}, \mathbf{h})$ combination.

Lazy dimensionality reduction. L-NAMOA^{*}_{dr}-MVH also uses dimensionality reduction for local dominance checks, i.e., when determining whether a path reaching a state s is dominated by another path reaching the same state.

A path reaching s with cost $\mathbf{g}$ is first tested against $G_{cl}^{Tr}(s)$. When the ordering condition required for dimensionality reduction holds, this truncated dominance test is sufficient. Otherwise, the algorithm falls back to a full dominance test against $G_{cl}(s)$. This fallback preserves correctness even when the MVH is admissible but inconsistent.

Computational bottleneck. Although the lazy representation avoids explicitly constructing $F(s, \mathbf{g})$, CHOOSEH may still examine many heuristic vectors. In the original algorithm, $H(s)$ is scanned sequentially in lexicographic order. For every candidate $\mathbf{h}$, the algorithm must determine whether $\text{Tr}(\mathbf{g} + \mathbf{h})$ is dominated by the current set T . Consequently, when $H(s)$ contains many heuristic vectors, a large fraction of the set may be inspected repeatedly for different paths reaching the same state.

Our approach targets this operation. Rather than changing the search order or the lazy heuristic-selection policy, we use a geometric index over $\text{Tr}(H(s))$ to identify and skip groups of heuristic vectors whose evaluation vectors are known to be dominated.

5 KD-Tree Accelerated Heuristic Selection

We replace the sequential heuristic scan performed by CHOOSEH with a geometric pruning procedure over the truncated heuristic vectors $\text{Tr}(H(s))$. For each state s , we construct a static KD-tree over these vectors. The tree is used only as a pruning index: it identifies groups of heuristic vectors that can be classified together, while the original lexicographic ordering of $H(s)$ continues to determine which heuristic is selected. Consequently, KD-CHOOSEH preserves the heuristic-selection rule of the original CHOOSEH procedure, although the KD-tree pruning mechanism itself does not require this ordering.

5.1 KD-Tree Representation

For every state s , let

$$H(s) = \langle \mathbf{h}_1, \dots, \mathbf{h}_k \rangle$$

denote its heuristic vectors sorted in lexicographic order,

$$\mathbf{h}_1 <_{\text{lex}} \mathbf{h}_2 <_{\text{lex}} \dots <_{\text{lex}} \mathbf{h}_k.$$

Each heuristic vector retains its index in this ordering.

A separate KD-tree $\mathcal{K}(s)$ is constructed over

$$\{\text{Tr}(\mathbf{h}_1), \dots, \text{Tr}(\mathbf{h}_k)\}.$$

Since $H(s)$ is fixed throughout the search, $\mathcal{K}(s)$ is constructed only once and is shared by all search paths that reach state s . The same KD-tree can also be reused across multiple searches whenever the underlying heuristic set $H(s)$ remains unchanged.

For a KD-tree node N , let H_N denote the set of heuristic vectors contained in the subtree rooted at N . Each KD-tree node N stores two aggregate vectors, denoted $N.\text{min}$ and $N.\text{max}$, corresponding to the component-wise minimum and maximum of the truncated heuristic vectors in its subtree. Formally,

$$N.\text{min} = \left(\min_{\mathbf{h} \in H_N} h_1, \dots, \min_{\mathbf{h} \in H_N} h_d \right)$$

and

$$N.\text{max} = \left(\max_{\mathbf{h} \in H_N} h_1, \dots, \max_{\mathbf{h} \in H_N} h_d \right).$$

Thus, for every $\mathbf{h} \in H_N$,

$$N.\text{min} \preceq \text{Tr}(\mathbf{h}) \preceq N.\text{max}.$$

Each node N also stores the set $N.I$ of lexicographic indices of the heuristic vectors contained in its subtree.

The KD-tree itself is independent of the evolving goal frontier. As additional solutions are discovered, the same static tree is queried against a stronger dominance set, allowing progressively larger portions of $H(s)$ to be pruned without rebuilding the index.

5.2 KD-Tree Construction

The KD-tree operates in $d - 1$ dimensions. At each internal node, we select the coordinate having the largest spread and partition the vectors around the median of that coordinate. The recursion stops when a node contains at most B heuristic vectors. Heuristic vectors within a leaf are examined directly. This avoids paying the cost of additional tree traversal when only a small number of individual dominance tests remain. The parameter B therefore controls the tradeoff between finer geometric pruning and KD-tree traversal overhead.

5.3 Aggregate Dominance Tests

The KD-tree accelerates heuristic selection by using the aggregate bounds stored at each node to classify entire subtrees without examining their heuristic vectors individually.

Let $T = G_{\text{cl}}^{\text{Tr}}(s_{\text{goal}})$ be the current truncated goal frontier, and define

$$\text{Dom}(T, \mathbf{x}) \iff \exists \mathbf{t} \in T: \mathbf{t} \preceq \mathbf{x}.$$

For a path reaching s with cost $\mathbf{g}$, the KD-tree uses the aggregate vectors of a node N to classify its entire subtree.

Algorithm 2 BUILDKDTREE(X, B)

Input: A set $X = \{(\text{Tr}(\mathbf{h}_i), i)\}$ of truncated heuristic vectors and their lexicographic indices; leaf size B

Output: A KD-tree node N

```

1: Create a new node  $N$ 
2:  $N.I \leftarrow \{i \mid (\text{Tr}(\mathbf{h}_i), i) \in X\}$ 
3:  $N.\text{min} \leftarrow \min_{(\mathbf{x}, i) \in X} \mathbf{x} \triangleright$  component-wise minimum
4:  $N.\text{max} \leftarrow \max_{(\mathbf{x}, i) \in X} \mathbf{x} \triangleright$  component-wise maximum
5: if  $|X| \leq B$  then
6:    $N.\text{leaf} \leftarrow \text{true}$ 
7:    $N.X \leftarrow X$ 
8:   return  $N$ 
9:  $j \leftarrow \text{argmax}_{m \in \{1, \dots, d-1\}} (\max_{(\mathbf{x}, i) \in X} x_m - \min_{(\mathbf{x}, i) \in X} x_m)$ 
    $\triangleright$  largest-spread dimension
10: Partition  $X$  around the median of coordinate  $j$  into  $X_{\text{left}}$ 
    and  $X_{\text{right}}$ 
11:  $N.\text{left} \leftarrow \text{BUILDKDTREE}(X_{\text{left}}, B)$ 
12:  $N.\text{right} \leftarrow \text{BUILDKDTREE}(X_{\text{right}}, B)$ 
13: return  $N$ 

```

Discarding an entire subtree. The vector $N.\text{min}$ is a component-wise lower bound on every truncated heuristic vector in the subtree. Suppose

$$\text{Dom}(T, \text{Tr}(\mathbf{g}) + N.\text{min}).$$

Since

$$\text{Tr}(\mathbf{g}) + N.\text{min} \preceq \text{Tr}(\mathbf{g} + \mathbf{h}) \quad \text{for all } \mathbf{h} \in H_N,$$

any goal vector that dominates $\text{Tr}(\mathbf{g}) + N.\text{min}$ also dominates every heuristic evaluation in the subtree. Therefore, all heuristic vectors in the subtree are dominated, and the entire subtree can be discarded without examining its vectors individually.

Accepting an entire subtree. Conversely, $N.\text{max}$ is a component-wise upper bound on every truncated heuristic vector in the subtree. Suppose

$$\neg \text{Dom}(T, \text{Tr}(\mathbf{g}) + N.\text{max}).$$

Since

$$\text{Tr}(\mathbf{g} + \mathbf{h}) \preceq \text{Tr}(\mathbf{g}) + N.\text{max} \quad \text{for all } \mathbf{h} \in H_N,$$

any goal vector that dominated one of these heuristic evaluations would also dominate $\text{Tr}(\mathbf{g}) + N.\text{max}$. Therefore, none of the heuristic vectors in the subtree is dominated, and the entire subtree can be accepted without examining its vectors individually.

When neither aggregate test is conclusive, the query recursively examines the children of N .

5.4 KD-Tree Heuristic Selection

Algorithm 3 replaces the linear CHOOSEH procedure of L-NAMOA^{*}_{dr}-MVH. It first uses the KD-tree to identify all heuristic vectors that are dominated by the current goal frontier. It then returns the lexicographically first heuristic vector that was not eliminated.

The recursive KD-tree query is shown in Algorithm 4.

Algorithm 3 KD-CHOOSEH($s, \mathbf{g}, T$)

Input: State s , path cost $\mathbf{g}$, truncated goal frontier T

Output: Lexicographically smallest usable heuristic, or $\perp$

```
1:  $D \leftarrow \emptyset$   $\triangleright$  indices certified as dominated
2: KD-PRUNE( $\mathcal{K}(s).root, \mathbf{g}, T, D$ )
3: for  $i = 1, \dots, |H(s)|$  do
4:   if  $i \notin D$  then
5:     return  $\mathbf{h}_i$ 
6: return  $\perp$ 
```

Algorithm 4 KD-PRUNE($N, \mathbf{g}, T, D$)

Input: KD-tree node N , path cost $\mathbf{g}$, goal frontier T , dominated-index set D

```
1:  $\mathbf{q}^{\min} \leftarrow \text{Tr}(\mathbf{g}) + N.\min$ 
2: if Dom( $T, \mathbf{q}^{\min}$ ) then
3:    $D \leftarrow D \cup N.I$   $\triangleright$  all vectors in the subtree are dominated
4:   return
5:  $\mathbf{q}^{\max} \leftarrow \text{Tr}(\mathbf{g}) + N.\max$ 
6: if not Dom( $T, \mathbf{q}^{\max}$ ) then
7:   return  $\triangleright$  no vector in the subtree is dominated
8: if  $N.\text{leaf}$  then
9:   for all  $(\text{Tr}(\mathbf{h}_i), i) \in N.X$  do
10:     $\mathbf{q} \leftarrow \text{Tr}(\mathbf{g}) + \text{Tr}(\mathbf{h}_i)$ 
11:    if Dom( $T, \mathbf{q}$ ) then
12:       $D \leftarrow D \cup \{i\}$ 
13:   return
14: KD-PRUNE( $N.\text{left}, \mathbf{g}, T, D$ )
15: KD-PRUNE( $N.\text{right}, \mathbf{g}, T, D$ )
```

Replacing CHOOSEH with KD-CHOOSEH leaves the remainder of L-NAMOA_{dr}-MVH unchanged, including the OPEN ordering, lazy re-evaluation mechanism, and local dominance checks. We formally show in Section 6 that KD-CHOOSEH returns the same heuristic vector as the original CHOOSEH procedure, thereby preserving the behavior and correctness guarantees of the original algorithm.

5.5 Numerical Example

Consider a problem with three objectives and let the KD-tree leaf size be $B = 3$. After dimensionality reduction, dominance tests operate in two dimensions. Suppose a path reaches state s with

$$\text{Tr}(\mathbf{g}) = (10, 10),$$

and assume that $H(s)$ contains six heuristic vectors $\mathbf{h}_1, \dots, \mathbf{h}_6$, appearing in this order in the lexicographic ordering of $H(s)$. Their truncated values are

$$\begin{aligned} \text{Tr}(\mathbf{h}_1) &= (2, 9), & \text{Tr}(\mathbf{h}_2) &= (3, 8), & \text{Tr}(\mathbf{h}_3) &= (4, 7), \\ \text{Tr}(\mathbf{h}_4) &= (8, 3), & \text{Tr}(\mathbf{h}_5) &= (9, 2), & \text{Tr}(\mathbf{h}_6) &= (11, 1). \end{aligned}$$

We first construct the root of the KD-tree according to Algorithm 2. The spread of the first truncated coordinate is $11 - 2 = 9$, whereas the spread of the second coordinate is $9 - 1 = 8$. The first coordinate therefore has the largest

spread and is selected as the splitting dimension. The median partition separates the three vectors with the smallest values in this coordinate from the three with the largest values, giving two leaf nodes,

$$H_{N_L} = \{\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3\}, \quad H_{N_R} = \{\mathbf{h}_4, \mathbf{h}_5, \mathbf{h}_6\}.$$

Assume that the current truncated goal frontier consists of the single vector

$$T = \{(12, 17)\}.$$

We first query the root node N . Its aggregate vectors are

$$N.\min = (2, 1), \quad N.\max = (11, 9).$$

The minimum-aggregate evaluation is

$$\text{Tr}(\mathbf{g}) + N.\min = (12, 11).$$

Since (12, 17) does not dominate (12, 11), the minimum test cannot certify that the entire root is dominated. We therefore consider its maximum aggregate,

$$\text{Tr}(\mathbf{g}) + N.\max = (21, 19).$$

This vector is dominated by (12, 17), so the maximum test cannot certify that the entire root is non-dominated either. The root is therefore inconclusive, and the query proceeds to its children.

For the left subtree N_L , the aggregate vectors are

$$N_L.\min = (2, 7), \quad N_L.\max = (4, 9).$$

Its minimum-aggregate evaluation is

$$\text{Tr}(\mathbf{g}) + N_L.\min = (12, 17).$$

Since $(12, 17) \preceq (12, 17)$, the minimum-aggregate test certifies that every heuristic vector in N_L is dominated. Indeed,

$$\begin{aligned} \text{Tr}(\mathbf{g} + \mathbf{h}_1) &= (12, 19), \\ \text{Tr}(\mathbf{g} + \mathbf{h}_2) &= (13, 18), \\ \text{Tr}(\mathbf{g} + \mathbf{h}_3) &= (14, 17), \end{aligned}$$

and (12, 17) weakly dominates all three. Hence, the entire subtree N_L can be discarded without examining its heuristic vectors individually.

Now consider the right subtree N_R , whose aggregate vectors are

$$N_R.\min = (8, 1), \quad N_R.\max = (11, 3).$$

Its minimum-aggregate evaluation is

$$\text{Tr}(\mathbf{g}) + N_R.\min = (18, 11).$$

The goal vector (12, 17) does not dominate (18, 11), so the subtree cannot be discarded. We therefore evaluate its maximum aggregate,

$$\text{Tr}(\mathbf{g}) + N_R.\max = (21, 13).$$

Again, (12, 17) does not dominate this vector, because $17 > 13$. The maximum-aggregate test therefore certifies that none of the heuristic vectors in N_R is dominated by the current goal frontier, and the entire subtree can be accepted without examining its heuristic vectors individually.

After the geometric query, the KD-tree has therefore classified $\{\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3\}$ as dominated and $\{\mathbf{h}_4, \mathbf{h}_5, \mathbf{h}_6\}$ as non-dominated.

Since heuristic selection follows the original lexicographic ordering, KD-CHOOSEH returns $\mathbf{h}_4$, the first surviving heuristic in $H(s)$. This is exactly the heuristic that the original sequential CHOOSEH procedure would return. However, the KD-tree reaches this decision through aggregate tests on N , N_L , and N_R , without individually testing the six heuristic vectors.

6 Theoretical Analysis

7 Empirical Evaluation

References

Wolff, M.; Felner, A.; and Salzman, O. 2026. Bridging Multi-Valued Heuristics and Dimensionality Reduction in Multi-Objective Search. In *Proceedings of the International Symposium on Combinatorial Search (SoCS)*.